

Assignment 1

Julia Nething
nething@gatech.edu

1 RESEARCH LOG

1.1 Background

I wanted to take Educational Technology because I felt that it would help me in my day job, where I teach at a software engineering immersive (Hack Reactor). At Hack Reactor, we teach JavaScript fundamentals, algorithms (LeetCode-style questions), and full-stack web development. I have found that algorithms, in particular problems using recursion, are the hardest to teach. We can give them sample problems and do demonstrations but eventually we just tell the students to keep practicing. This is not effective; students can't practice what they don't know how to do in the first place. My goal for this project is to research and understand the various strategies for teaching these types of problems, develop a curriculum that teaches each of these strategies, and give a quiz or reflective survey before and after to determine which strategies are the most useful for the most students so we can focus on those in the future. I think recursion would be a good starting point because that is a common point of misunderstanding, but I am interested in reading about a variety of strategies for teaching algorithm-style questions to see if there is a better starting point. Additionally, I am interested in reading about teaching related topics, such as skills for breaking down problems and for debugging.

1.2 Papers

1.2.1 *A Hybrid Model for Teaching Recursion*

Omar AlZoubi et al. (2015). "A Hybrid Model for Teaching Recursion". In: *Proceedings of the 16th Annual Conference on Information Technology Education*. SIGITE '15. Chicago, Illinois, USA: Association for Computing Machinery, pp. 65–70. ISBN: 9781450338356. DOI: [10.1145/2808006.2808030](https://doi.org/10.1145/2808006.2808030). URL: <https://doi.org/10.1145/2808006.2808030>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "teaching recursion".

In this study, professors at Carnegie Mellon University developed ChiQat-Tutor,

a program that allowed students to try several strategies for learning recursion. The students were also given a pre and post test to measure their increase in learning. There was a small sample size (15 students) but it seemed to help the students. Some of the strategies were having the students trace a graph through a recursive call, fixing a recursive graph, and constructing their own recursive graph (AlZoubi et al., 2015).

I like that the researchers employed multiple strategies and conducted a pre and post test to measure learning. However, because there is a small sample size and the strategies were tested at the same time, it is difficult to tell which strategy was the most effective. Perhaps the main takeaway here is to combine multiple strategies and allow the students to gravitate towards the strategy that resonates with them the most.

1.2.2 A Structured Approach to Teaching Recursion Using Cargo-Bot

Elynn Lee et al. (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*. ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: [10.1145/2632320.2632356](https://doi.org/10.1145/2632320.2632356). URL: <https://doi.org/10.1145/2632320.2632356>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "teaching recursion".

The professors at University of Texas at Austin expanded upon a previous study that used Cargo-Bot, an iPad game for learning recursion. The game requires students move boxes from an original pattern into a goal pattern, and their only options for repetition are using recursion. This study involved approximately 200 undergraduate students who played the game and listened to lectures that included examples from the game. Results showed that students who practiced more than seven puzzles had statistically significantly higher scores than the control group (Lee et al., 2014).

It is interesting that students responded so well to the game and were able to then use that knowledge to read and write recursive code. It is also significant that repetition was so important; students who only completed seven puzzles did not have statistically significantly higher scores. I would be interested to see if other universities used similar games (preferably ones that are open-sourced

and not limited to an iPad).

1.2.3 Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi

Alan C. Benander and Barbara A. Benander (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi". English. In: *Journal of Information Systems Education* 19.4. Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455-467. URL: <https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107>

I found it via the previous paper's references (Lee et al., 2014).

In this paper, teachers trained students to do a live demonstration of the Tower of Hanoi problem, which is a problem that is best solved recursively. When a student demonstrator makes a new recursive call, they step back while another student steps forward to start the new call. While this is happening, the students watching the demonstration take notes about the values and which demonstrator was working at which times. Students self-reported that they all found the demonstration to be neutral or beneficial to their understanding of recursion (Benander and Benander, 2008).

This is an unusual approach to teaching recursion because no code was actually involved. Also, most students were watching and taking notes, not actively solving a problem. I would be interested to see if other universities used similar demonstrations and if there is any statistically significant data that it improved recursion coding ability.

1.2.4 Why Structural Recursion Should Be Taught Before Arrays in CS 1

Kim B Bruce, Andrea Danyluk, and Thomas Murtagh (2005). "Why structural recursion should be taught before arrays in CS 1". In: *SIGCSE bulletin*. 37.1, pp. 246-250. ISSN: 0097-8418

I found it via the previous paper's references (Benander and Benander, 2008).

Professors teaching an introductory computer science course found that moving the curriculum around so that students learn structural recursion before they

learn about data structures such as arrays led to overall better understanding of recursion. They presented structural recursion, for example moving through the rings of a bulls-eye, which they found to be more intuitive to students. Students gave the class lower difficulty scores than students in previous semesters of the same class (Bruce, Danyluk, and Murtagh, 2005).

I like their idea to present structural recursion (like the bulls-eye) instead of typical numeric recursion. It is also interesting that by showing the students recursion earlier on, it made everything seem easier by comparison. I am interested in learning more about how the order of topics affects overall understanding.

1.2.5 Improving Debugging Skills in the Classroom: The Effects of Teaching a Systematic Debugging Process

Tilman Michaeli and Ralf Romeike (2019). "Improving Debugging Skills in the Classroom: The Effects of Teaching a Systematic Debugging Process". In: *Proceedings of the 14th Workshop in Primary and Secondary Computing Education*. WiP-SCE'19. Glasgow, Scotland, UK: Association for Computing Machinery. ISBN: 9781450377041. DOI: [10.1145/3361721.3361724](https://doi.org/10.1145/3361721.3361724). URL: <https://doi.org/10.1145/3361721.3361724>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "teaching debugging".

In this study, professors divided their students into two groups. Both groups were given a pre and post test and debugging tasks for code they were already familiar with. However, one group was given an lesson where they were taught about different error types and how to systematically approach each one, while the other group was not. They found that students who were given the lecture did better on the debugging exercises (Michaeli and Romeike, 2019).

It is interesting to me that even just one short lecture on debugging process can make a difference, especially when you consider that the instructions seem pretty obvious (read the error message, for example). I like that the professors broke down different bug types, because that does seem to be a common misunderstanding for students. I would be interested if changing the format of the debugging tasks would help even more.

1.2.6 Debugging Students' Debugging Process

Axel Bottcher et al. (2016). "Debugging students' debugging process". eng. In: *2016 IEEE Frontiers in Education Conference (FIE)*. IEEE, pp. 1–7. ISBN: 9781509017904

I found this via the Georgia Tech digital library when I searched for "students debugging".

Professors wanted to teach their undergraduate students better debugging skills. They created a Java program that had several specific bugs, and then students were required to document their debugging process thoroughly (including screenshots of their code paused at a debugger, hypotheses for what would happen, and reflection on what caused the error). They found that the wide variety of results gave them more insight into how their students worked and the skills they needed to develop in order to become better programmers (following directions, for example) (Bottcher et al., 2016).

I really like this exercise because it forces students to slow down (in order to hypothesize, take screenshots, etc.). The wide variety of results shows that many students are not working systematically and following directions. I would like to read some other papers where the professors aim to teach core skills and common problems, such as effective debugging.

1.2.7 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data

Amjad Altadmri and Neil C.C. Brown (2015). "37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data". In: *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. SIGCSE '15. Kansas City, Missouri, USA: Association for Computing Machinery, pp. 522–527. ISBN: 9781450329668. DOI: [10.1145/2676723.2677258](https://doi.org/10.1145/2676723.2677258). URL: <https://doi.org/10.1145/2676723.2677258>

I found it via another paper's references (Michaeli and Romeike, 2019).

The Blackbox data set comprises the errors of 265,000 Java users around the world. This paper analyzes those errors to see which errors are the most frequent, take the longest to fix, etc. The errors that take the longest for users to fix are the ones that do not throw a compiler error; for example, forgetting to use the return keyword (Altadmri and Brown, 2015).

This is extremely relevant to the recursion and debugging topics, because it shows that students have a hard time debugging semantic bugs like those found in recursive algorithms. This paper does not offer any solutions to the problem, however it confirms at a larger scale a suspicion that I had from teaching at a smaller scale. This paper also provides a useful category hierarchy for bugs which may be helpful for students to see so they can better identify bugs in the future.

1.2.8 *Teaching Recursion Using Recursion Graphs*

Wen-Jung Hsin (Apr. 2008). "Teaching Recursion Using Recursion Graphs". In: 23.4, pp. 217–222. ISSN: 1937-4771

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

In this paper, the author proposes using an RGraph (Recursion Graph) to teach students about recursion. In an RGraph, items in squares are recursive calls, items in circles are processing steps, solid arrows indicate items in the same step, and dashed arrows indicate items that flow to a previous or next step. The author demonstrates two recursive functions this way, a rabbit problem and the Towers of Hanoi problem (Hsin, 2008).

RGraphs have been referenced in other papers so it was good to finally understand what they mean and represent. I have a hard time following the graph but I do think that some students will resonate with it. I am interested to look for more papers that attempt to diagram recursive calls.

1.2.9 *Evaluating the Tracing of Recursion in the Substitution Notional Machine*

Preston Tunnell Wilson, Kathi Fisler, and Shriram Krishnamurthi (2018). "Evaluating the Tracing of Recursion in the Substitution Notional Machine". In: *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*. SIGCSE '18. Baltimore, Maryland, USA: Association for Computing Machinery, pp. 1023–1028. ISBN: 9781450351034. DOI: [10.1145/3159450.3159479](https://doi.org/10.1145/3159450.3159479). URL: <https://doi.org/10.1145/3159450.3159479>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

In this paper, the authors gave students at a university a different quiz each

week that built upon the ideas of substitution in recursion. The quizzes asked the students to write down the order of execution and substitute the values or code in order. The authors found that this helped students be able to read difficult recursion problems later on, however due to the amount of writing it also encouraged students to take shortcuts and make mistakes (Tunnell Wilson, Fisler, and Krishnamurthi, 2018).

This was interesting to me because this is also how I teach students recursion, but I teach them substitution for writing the code not for reading the code. I think this could be a good way to introduce students to methodical thinking, similar to the debugging exercises in other papers. I like that they used repeatable exercises done over several months with the same students and would like to find similar papers.

1.2.10 On Teaching Recursion

Robert L. Kruse (Feb. 1982). "On Teaching Recursion". In: *SIGCSE Bull.* 14.1, pp. 92–96. ISSN: 0097-8418. DOI: [10.1145/953051.801346](https://doi.org/10.1145/953051.801346). URL: <https://doi.org/10.1145/953051.801346>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The author states that tree diagrams are a great way to teach students about recursive calls. The author also states that it can help students understand when recursion is not beneficial, because in those cases the recursive tree will look more like a straight line (these cases can be reformatted to use a simple loop instead). The paper shows several examples of how values can be passed down in the tree diagram to help students follow the path of execution (Kruse, 1982).

This is a very simple way of showing recursion and something that I think a lot of students would benefit from doing. I wonder if this could be combined with other methods, like the code substitution paper from earlier (so each node in the tree would also include code along with values). This paper does not test students or try to evaluate its effectiveness in any way, so I would like to read papers that do evaluate impact.

1.2.11 *Teaching Recursion Using Fractals in Prolog*

Bruce S. Elenbogen and Martha R. O’Kennon (Feb. 1988). “Teaching Recursion Using Fractals in Prolog”. In: *SIGCSE Bull.* 20.1, pp. 263–266. ISSN: 0097-8418. DOI: [10.1145/52965.53029](https://doi.org/10.1145/52965.53029). URL: <https://doi.org/10.1145/52965.53029>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The author states that the Prolog language and visual fractals are good tools for teaching programming. In the paper, they give examples of series of drawings (snowflakes, for example) that use recursion, and then the subsequent Prolog code (or pseudocode). The author thinks that pairing the drawings with code allows students to get more than one perspective of recursion (Elenbogen and O’Kennon, 1988).

I do like the drawings of the snowflakes and think that that is a good way to show a student who has never heard of recursion what it is at a basic level. However, actually programming the recursive drawing is complicated because the drawings are very detailed. I also would like to see more papers that try to test whether students found the technique helpful or not.

1.2.12 *An Implementation-Independent Approach to Teaching Recursion*

Gary Ford (Jan. 1984). “An Implementation-Independent Approach to Teaching Recursion”. In: *SIGCSE Bull.* 16.1, pp. 213–216. ISSN: 0097-8418. DOI: [10.1145/952980.808655](https://doi.org/10.1145/952980.808655). URL: <https://doi.org/10.1145/952980.808655>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The author presents an approach for teaching recursion that is based in mathematics (mathematical induction) and not in any programming language. They argue that teaching recursion should be more similar to teaching basic if statements in the sense that they are not language-specific but taught at a higher level. The paper includes some examples that essentially just show the objective, parameters for the function, and high-level pseudocode (Ford, 1984).

While this is a valid approach for teaching computer science, I think this is more applicable to students who are coming from other math or science disciplines.

The students I teach usually do not have these types of backgrounds, so this would not resonate with them as much. As I have stated with other papers, I would like to have seen some pre and post testing to see how students benefit from this strategy.

1.2.13 *Conceptual Models and Cognitive Learning Styles in Teaching Recursion*

Cheng-Chih Wu, Nell B. Dale, and Lowell J. Bethel (Mar. 1998). "Conceptual Models and Cognitive Learning Styles in Teaching Recursion". In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: [10.1145/274790.274315](https://doi.org/10.1145/274790.274315). URL: <https://doi.org/10.1145/274790.274315>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The authors split up undergraduate students into four groups, abstract versus concrete learners, then those split into abstract versus concrete examples. The concrete examples used Russian nesting dolls and diagram tracing, while the abstract examples used mathematical induction. They found that abstract learners and concrete examples (not necessarily together) resulted in higher scores on the post test (Wu, Dale, and Bethel, 1998).

I like that this class was big enough to divide students up by their learning style and that the professors were able to develop two lectures for this study. It is also interesting that the type of learner and the type of example that were the most beneficial were not the same; I would have thought that the type of example would depend on the type of learner. I would be interested to read more articles about how learning style determines the learning strategy.

1.2.14 *Teaching Recursion Using Recursively-Generated Geometric Designs*

Aaron Gordon (Oct. 2006). "Teaching Recursion Using Recursively-Generated Geometric Designs". In: *J. Comput. Sci. Coll.* 22.1, pp. 124–130. ISSN: 1937-4771

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The author argues that showing students recursive geometric shapes is a great way to teach them about recursion. The paper goes over four types of examples with corresponding code: triangles, polygons, fractals and tiling. There are no results showing whether or not this was proven to be beneficial to students

(Gordon, 2006).

This paper is very similar to the other fractal paper, but it goes over some different examples which might be useful for students. It also includes what looks like working code instead of general pseudocode. I would be interested to see if there is a paper that tests the efficacy of this strategy on students' understanding of recursion.

1.2.15 *Experimental Evaluation of Teaching Recursion in a Video Game*

Amanda Chaffin et al. (2009). "Experimental Evaluation of Teaching Recursion in a Video Game". In: *Proceedings of the 2009 ACM SIGGRAPH Symposium on Video Games*. New York, NY, USA: Association for Computing Machinery, pp. 79–86. ISBN: 9781605585147. URL: <https://doi.org/10.1145/1581073.1581086>

I found this via the Association for Computing Machinery (ACM) digital library when I searched for "recursion".

The authors tested a recursion game on 43 undergraduate students. The game involves writing recursive binary search code in order to solve puzzles. One level involves a game of telephone where characters call each other (recursively) to get information. Students self reported that they enjoyed the game; they also received higher post test scores after playing (Chaffin et al., 2009).

This is one of several studies that show that students enjoy visual representations of recursion and they enjoy games, even if the games involve coding. I would be interested to see if this game is open-sourced or available for non-students to play. I would also be interested in using the telephone example with students in the future.

1.3 Synthesis

After reading all of these options, I'm surprised both at the wide range of strategies that teachers have employed to teach recursion and debugging skills, and also how poorly tested the different strategies are. I expected to find at least one gold standard way of teaching these topics (for example from a well-documented study). Some common techniques appear to include lectures, games, visualizations of code execution, bug hunting, mathematical induction, and geometry/fractals, but there is no widely agreed upon way of teaching recursion.

The biggest takeaway seems to be that you need to provide a variety of strategies

to all students so they can practice in the way that makes the most sense to them. This was especially evident in studies that used multiple teaching techniques; it is difficult to tell which students are going to benefit from which strategies, so they should be allowed to try different types (AlZoubi et al., 2015) (Wu, Dale, and Bethel, 1998). If I do continue down this recursion path, I also need to make sure that I am testing whether or not the strategy was effective for students, because many of these papers are not effectively testing strategies.

I think this process is leading me away from attempting to teach recursion in all the ways possible, because there are simply too many ways. Instead, I think I should either choose a different programming topic altogether, or I should choose one strategy that is applicable to many recursive problems and create a polished tool around that strategy. For example, I could create a d3 visualization that works for many different recursive problems.

1.4 Reflection

This process as a whole was not very easy for me just because I do not have a lot of experience reading technical research papers. Thankfully I have used Google Scholar before so that was not very difficult to navigate, and thankfully many papers had good references that led me to other papers. However, I am not sure that I chose the best papers for my topic and I am also not confident that I synthesized them properly to gain the information I need moving forward. I am very interested to hear others' strategies for finding relevant papers that propel them to the next step.

Going forward, I am not sure that I am as interested in the recursion topic as I thought I was going into this week. One option for next week would be to focus more on one particular strategy such as using d3 to visualize recursion better. Otherwise, there are too many different recursion strategies and many of them require a lot of work to even get started (for example, I imagine the games took a lot of time to implement).

1.5 Planning

After reading these papers this week, I am not sure that recursion is a good starting place because I still lack a clear direction (though that might mean that it is a good starting place). I think that next week I would like to explore other themes in teaching programming, whether that entails debugging, data structures, other

types of algorithms, git, etc.

Another strategy could be to focus on a particular teaching strategy that is applicable to multiple topics, such as visualizations. In recent years especially, web development has evolved to include amazing libraries such as d3, and I would be interested to see how those libraries are being incorporated into classroom visualizations.

A final strategy could be to focus on how to effectively evaluate students after teaching a strategy. Many of the papers that I read this week either did not evaluate the strategy at all or did so with very small sample sizes. I would like to learn more about what is needed to evaluate a particular teaching strategy (sample size, pre and post testing, etc.).

I will think about all three of these this week and see which strategy offers the most viable content.

2 ACTIVITY

2.1 Debugging Students' Debugging Process

Axel Bottcher et al. (2016). "Debugging students' debugging process". eng. In: *2016 IEEE Frontiers in Education Conference (FIE)*. IEEE, pp. 1–7. ISBN: 9781509017904

2.1.1 Need

Students are not good at debugging their code because they don't know how. These professors aim to teach students a better debugging process.

2.1.2 Method

The authors created a debugging exercise for students. They wrote several specific bugs into code and then required the students to answer specific questions at each point in the process (hypothesize about where the bug is, explain the bug, take screenshots of them using a debugger, etc.).

2.1.3 Audience

The participants for this exercise were undergraduate students, but the audience is anyone who is a novice programmer or needs to strengthen their debugging process.

2.1.4 Results

They received a wide variety of results, from students who did not follow directions to students who fully followed the assignment. They found that students who did follow the assignment and used technical language throughout did well on the final exam (no one who used technical language on this assignment failed the final).

2.1.5 Critique

The results are interesting but may be revealing more correlation than causation. It is not clear that the debugging exercise improved anyone's scores on later exams, but rather that students who do well on the assignment also do well on later exams. I would like to see the students complete the same debugging assignment over the course of the semester and see if improved scores on the debugging assignment are correlated to better exam scores as well.

2.2 A Hybrid Model for Teaching Recursion

Omar AlZoubi et al. (2015). "A Hybrid Model for Teaching Recursion". In: *Proceedings of the 16th Annual Conference on Information Technology Education*. SIGITE '15. Chicago, Illinois, USA: Association for Computing Machinery, pp. 65–70. ISBN: 9781450338356. DOI: [10.1145/2808006.2808030](https://doi.org/10.1145/2808006.2808030). URL: <https://doi.org/10.1145/2808006.2808030>

2.2.1 Need

Students have trouble understanding recursion for a variety of reasons (it is difficult to follow, they don't have a proper mental model for it, etc.).

2.2.2 Method

The authors built ChiQat-Tutor, a program that allows students to walk through a few exercises to practice recursion. Some examples of these exercises include fixing bugs in existing code, tracing through a recursive call stack, writing code, and answering multiple-choice questions.

2.2.3 Audience

The participants in this study were sixteen undergraduate students at Carnegie Mellon University in Qatar, but the exercises could be useful for any novice

programmer.

2.2.4 Results

They found that there was a statistically significant increase in scores between the pre test and the post test. However, it is unclear which particular exercises were the most beneficial.

2.2.5 Critique

The need and the method seem to be aligned pretty well, but unfortunately the sample size for this was too small (sixteen students). In addition, the different tactics were not separated out into different test groups, so it is impossible to know if a particular exercise was most effective or if it was the combination of the exercises that was effective.

2.3 Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi

Alan C. Benander and Barbara A. Benander (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi". English. In: *Journal of Information Systems Education* 19.4. Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455-467. URL: <https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107>

2.3.1 Need

Students have a difficult time understanding recursion and they need to see a visual representation of the path of execution.

2.3.2 Method

The authors taught student volunteers how to demonstrate recursion on a physical Towers of Hanoi set. Each student when it was their turn would step forward, and when they passed on the recursive call to someone else they would step back. Meanwhile, students watching would take notes on the values and how long each demonstrator worked.

2.3.3 Audience

The participants of this study were twelve undergraduate students but the exercise would be useful for any class of novice programmers.

2.3.4 Results

The students self-reported that they understood recursion better after they watched or participated in the demonstration.

2.3.5 Critique

These results are of a very small sample size and also rely on self-reporting, which is unreliable. Also, students may feel that they understand the concept better but still not be able to implement recursive code. The students were given a recursive problem to write code for after the demonstration, but there is no pre test to compare it to. I would like to see this same exercise performed with more students and with a pre and post test to see if students really did learn something from the demonstration.

2.4 A Structured Approach to Teaching Recursion Using Cargo-Bot

Elynn Lee et al. (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*. ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: [10.1145/2632320.2632356](https://doi.org/10.1145/2632320.2632356). URL: <https://doi.org/10.1145/2632320.2632356>

2.4.1 Need

Recursion is a difficult topic for students to learn. The authors wanted to solve this problem in a way that is fun and interactive but also self-paced.

2.4.2 Method

The authors used an iPad game called Cargo-Bot to teach recursion. The users have to move the boxes into a goal configuration using recursive techniques. The authors supplemented these self-paced puzzles with lectures with examples from the game.

2.4.3 Audience

The participants of this study were 204 undergraduate students, but it is applicable to any novice programmer.

2.4.4 Results

The authors found that students who did 8 or more puzzles in Cargo-Bot had statistically significant higher scores on later exams compared to the control group. They also reported enjoying the game but were not confident in their skill at playing the game.

2.4.5 Critique

This study had a much higher number of participants than other studies that I read, but 204 students is still not a lot. It is also unclear if the game was helpful compared to similar quantity of time performing a different recursion review exercise (for example, drawing a recursion graph). This study would need to be repeated with more students (and more variety of control groups) and a pre and post test to accurately determine if the Cargo-Bot game is useful.

2.5 Conceptual Models and Cognitive Learning Styles in Teaching Recursion

Cheng-Chih Wu, Nell B. Dale, and Lowell J. Bethel (Mar. 1998). "Conceptual Models and Cognitive Learning Styles in Teaching Recursion". In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: [10.1145/274790.274315](https://doi.org/10.1145/274790.274315). URL: <https://doi.org/10.1145/274790.274315>

2.5.1 Need

Students find learning recursion to be very difficult and the authors want to know if it is better to use abstract examples or concrete examples, and if that is dependent on the student's learning style.

2.5.2 Method

The authors had the students take a test to determine if they had a more abstract or concrete learning style. Then they were randomly distributed into four groups for the different combinations of abstract versus concrete learners and examples. The concrete example group was given Russian nesting dolls and diagram tracing, while the abstract example group was given mathematical induction. The

students were given a pre and post test.

2.5.3 Audience

The participants of this study were 237 undergraduate students but this is applicable to all novice programmers.

2.5.4 Results

The pre and post test revealed that concrete examples helped students the most, but abstract learners learned the most. The authors were surprised that the type of example did not match the type of learner.

2.5.5 Critique

This was a bigger sample size than most of the other papers I read for this assignment, but I still wish that the sample was bigger or that they had repeated the experiment. Also, I wonder if the test given to students is accurate or if it would have been more accurate if students chose which types of examples they wanted. Overall this seems to be a well-done study because they tried to control for the learning style and they had a bigger sample size than most of the other studies.

3 REFERENCES

- [1] Altadmri, Amjad and Brown, Neil C.C. (2015). "37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data". In: *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. SIGCSE '15. Kansas City, Missouri, USA: Association for Computing Machinery, pp. 522–527. ISBN: 9781450329668. DOI: [10.1145/2676723.2677258](https://doi.org/10.1145/2676723.2677258). URL: <https://doi.org/10.1145/2676723.2677258>.
- [2] AlZoubi, Omar, Fossati, Davide, Di Eugenio, Barbara, Green, Nick, Alizadeh, Mehrdad, and Harsley, Rachel (2015). "A Hybrid Model for Teaching Recursion". In: *Proceedings of the 16th Annual Conference on Information Technology Education*. SIGITE '15. Chicago, Illinois, USA: Association for Computing Machinery, pp. 65–70. ISBN: 9781450338356. DOI: [10.1145/2808006.2808030](https://doi.org/10.1145/2808006.2808030). URL: <https://doi.org/10.1145/2808006.2808030>.
- [3] Benander, Alan C. and Benander, Barbara A. (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi". English. In: *Journal of Information Systems Education* 19.4.

Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455-467. URL: <https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107>.

- [4] Bottcher, Axel, Thurner, Veronika, Schlierkamp, Kathrin, and Zehetmeier, Daniela (2016). "Debugging students' debugging process". eng. In: *2016 IEEE Frontiers in Education Conference (FIE)*. IEEE, pp. 1-7. ISBN: 9781509017904.
- [5] Bruce, Kim B, Danyluk, Andrea, and Murtagh, Thomas (2005). "Why structural recursion should be taught before arrays in CS 1". In: *SIGCSE bulletin*. 37.1, pp. 246-250. ISSN: 0097-8418.
- [6] Chaffin, Amanda, Doran, Katelyn, Hicks, Drew, and Barnes, Tiffany (2009). "Experimental Evaluation of Teaching Recursion in a Video Game". In: *Proceedings of the 2009 ACM SIGGRAPH Symposium on Video Games*. New York, NY, USA: Association for Computing Machinery, pp. 79-86. ISBN: 9781605585147. URL: <https://doi.org/10.1145/1581073.1581086>.
- [7] Elenbogen, Bruce S. and O'Kennon, Martha R. (Feb. 1988). "Teaching Recursion Using Fractals in Prolog". In: *SIGCSE Bull.* 20.1, pp. 263-266. ISSN: 0097-8418. DOI: [10.1145/52965.53029](https://doi.org/10.1145/52965.53029). URL: <https://doi.org/10.1145/52965.53029>.
- [8] Ford, Gary (Jan. 1984). "An Implementation-Independent Approach to Teaching Recursion". In: *SIGCSE Bull.* 16.1, pp. 213-216. ISSN: 0097-8418. DOI: [10.1145/952980.808655](https://doi.org/10.1145/952980.808655). URL: <https://doi.org/10.1145/952980.808655>.
- [9] Gordon, Aaron (Oct. 2006). "Teaching Recursion Using Recursively-Generated Geometric Designs". In: *J. Comput. Sci. Coll.* 22.1, pp. 124-130. ISSN: 1937-4771.
- [10] Hsin, Wen-Jung (Apr. 2008). "Teaching Recursion Using Recursion Graphs". In: 23.4, pp. 217-222. ISSN: 1937-4771.
- [11] Kruse, Robert L. (Feb. 1982). "On Teaching Recursion". In: *SIGCSE Bull.* 14.1, pp. 92-96. ISSN: 0097-8418. DOI: [10.1145/953051.801346](https://doi.org/10.1145/953051.801346). URL: <https://doi.org/10.1145/953051.801346>.
- [12] Lee, Elynn, Shan, Victoria, Beth, Bradley, and Lin, Calvin (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*.

- ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: [10 . 1145 / 2632320 . 2632356](https://doi.org/10.1145/2632320.2632356). URL: <https://doi.org/10.1145/2632320.2632356>.
- [13] Michaeli, Tilman and Romeike, Ralf (2019). “Improving Debugging Skills in the Classroom: The Effects of Teaching a Systematic Debugging Process”. In: *Proceedings of the 14th Workshop in Primary and Secondary Computing Education*. WiPSCE'19. Glasgow, Scotland, UK: Association for Computing Machinery. ISBN: 9781450377041. DOI: [10 . 1145 / 3361721 . 3361724](https://doi.org/10.1145/3361721.3361724). URL: <https://doi.org/10.1145/3361721.3361724>.
- [14] Tunnell Wilson, Preston, Fisler, Kathi, and Krishnamurthi, Shriram (2018). “Evaluating the Tracing of Recursion in the Substitution Notional Machine”. In: *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*. SIGCSE '18. Baltimore, Maryland, USA: Association for Computing Machinery, pp. 1023–1028. ISBN: 9781450351034. DOI: [10 . 1145 / 3159450 . 3159479](https://doi.org/10.1145/3159450.3159479). URL: <https://doi.org/10.1145/3159450.3159479>.
- [15] Wu, Cheng-Chih, Dale, Nell B., and Bethel, Lowell J. (Mar. 1998). “Conceptual Models and Cognitive Learning Styles in Teaching Recursion”. In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: [10 . 1145 / 274790 . 274315](https://doi.org/10.1145/274790.274315). URL: <https://doi.org/10.1145/274790.274315>.